



Vulnerability Assessment and Penetration Testing Report

**Damn Vulnerable Web Application (DVWA)
Security Assessment**

Submitted by

PALLAPU MAHESH

1EP22EC074

Submitted to

External Guide,

Mr.Jacob Dennis P,

Campuspe ,Bangalore.



Client	Metasploitable 2 Test Environment
Application	DVWA (Damn Vulnerable Web Application)
Report Date:	MAY 9-2026
Prepared By	PALLAPU MAHESH
Submitted to	Campuspe support team
Role	Security Reviewer
URL:	http://metaspliotable2ip/dvwam/

CONTENT

1. Executive Summary
2. Objectives
3. Scope of Assessment
4. Testing Methodology
5. Tools
6. Vulnerability Details
 - 6.1
 - 6.2
 - 6.3
 - 6.4
 - 6.5
 - 6.6
 - 6.7
 - 7.8
 - 7.9
- 8.

1. Executive Summary

A Vulnerability Assessment and Penetration Testing (VAPT) assessment was conducted against the Damn Vulnerable Web Application (DVWA) hosted on the Metasploitable 2 Lab virtual machine.

The objective of the assessment was to identify security weaknesses, validate exploitable vulnerabilities, and analyze the overall security posture of the target application.

The assessment identified multiple vulnerabilities including:

- SQL Injection
- Cross-Site Scripting (XSS)
- Command Injection
- Missing Security Headers
- Weak Cookie Security Configuration
- Absence of CSRF Protection
- Information Disclosure
- Remote Code Execution Exposure

The testing demonstrated that insecure input handling and outdated server configurations can expose applications to severe security risks.

2. Objectives

The objectives of this assessment were:

- Perform reconnaissance and service enumeration
- Identify web application vulnerabilities
- Validate vulnerabilities through controlled testing
- Analyze risk and impact
- Recommend mitigation strategies
- Demonstrate practical cybersecurity assessment methodology
-

Overall Risk Level: HIGH

The assessment **identified 13** security findings across multiple severity levels. Of significant concern are **2 High** severity vulnerabilities that pose substantial risk to the target application and underlying infrastructure.

These vulnerabilities could allow attackers to:

- Execute arbitrary commands on the web server
- Access sensitive application data
- Exploit weak session and cookie security
- Perform Cross-Site Request Forgery (CSRF) attacks
- Gather server information for further exploitation
- Bypass browser-side security protections

Severity	Count	Percentage	CVSS Range	Priority
Critical	0	0%	9.0-10.0	P0 - Immediate
High	2	15%	7.0-8.9	P1 - Urgent
Medium	3	43%	4.0 – 6.9	P2 - Importan
low	6	26%	0.1 – 3.9	P3 - Standard
Informational	2	16%	-	Monitoring
Total	13	100%	-	-

Critical Observations

High Severity Findings

- Remote Code Execution (CVE-2012-1823)
- Source Code Disclosure

Medium Severity Findings

- Absence of Anti-CSRF Tokens
- Missing Content Security Policy (CSP)
- Missing Anti-Clickjacking Protection

Low Severity Findings

- Cookie Without Http Only Flag
- Cookie Without Same Site Attribute
- Server Version Information Disclosure
- X-Content-Type-Options Header Missing
- Information Leakage via HTTP Headers

Recommendations

Immediate (0–24 Hours)

- Patch vulnerable PHP CGI components affected by CVE-2012-1823
- Disable unnecessary server information disclosure
- Restrict execution of untrusted input

Short-Term (1–7 Days)

- Implement CSRF protections across forms
- Configure Content Security Policy (CSP)
- Enable X-Frame-Options and X-Content-Type-Options headers

Long-Term

- Perform regular VAPT assessments
- Harden server configuration following OWASP standards
- Implement secure cookie attributes and session management controls

3. Scope of Assessment

Target Environment

Item	Details
Target Application	DVWA (Damn Vulnerable Web Application)
Target Platform	Metasploitable 2
Target IP Address	192.168.142.129
Testing Environment	Isolated Virtual Lab
Operating System	Ubuntu Linux
Web Server	Apache 2.2.8
Backend Language	PHP 5.2.4
Database	MySQL

In Scope

- Web application testing
- HTTP header analysis
- Vulnerability scanning
- Authentication analysis
- Input validation testing
- Manual and automated assessment

Out of Scope

- Denial of Service (DoS)
- Social Engineering
- Physical Security
- Real-world external targets
- Persistence or destructive exploitation

4. Testing Methodology

The assessment followed a structured VAPT methodology.

Phase 1 – Reconnaissance

Performed target discovery and service enumeration.

Activities:

- Port scanning
- Service detection
- Technology identification
- Web enumeration

Activities:

- SQL Injection testing
- XSS testing
- Command Injection testing
- Authentication analysis

5. Tools Used

Tool	Purpose
Kali Linux	Penetration testing platform
Nmap	Network reconnaissance and enumeration
Nikto	Web server vulnerability scanning
OWASP ZAP	Automated web application security scanning
Burp Suite	Web application request analysis
Firefox Browser	Web testing and validation

6. Vulnerability Details

6.1. Remote Code Execution – CVE-2012-1823 (High Risk)

Vulnerability

Remote Code Execution (RCE) allows an attacker to execute arbitrary system commands on the target server.

Affected Target

<http://192.168.142.129/dvwa/>

CVE

CVE-2012-1823

Risk Level

- Risk: High
- Confidence: Medium

Description

This vulnerability affects older PHP CGI configurations.

An attacker can manipulate query strings and inject PHP directives into the request, resulting in arbitrary command execution.

The scan indicates the server accepted malicious PHP parameters through the URL.

Evidence Found

w3gw669dqdfix39mhsc1

Attack Payload Observed

```
<?php exec('echo w3gw669dqdfix39mhsc1',$colm);echo join("", $colm);die();?>
```

Why It Is Dangerous

An attacker may:

- Execute OS commands
- Upload web shells
- Gain server access
- Read sensitive files
- Pivot deeper into the network
- Fully compromise the web application

Technical Cause

The server is running:

- Apache/2.2.8
- PHP/5.2.4

Older PHP CGI versions improperly process malformed query strings.

Remediation

- Upgrade PHP to a secure version
- Disable vulnerable CGI mode

- Use PHP-FPM instead of CGI
- Apply latest Apache security patches
- Configure ModSecurity WAF rules
- Restrict dangerous PHP functions:

`exec()`

`system()`

`shell_exec()`

- Validate and sanitize all inputs

6.2 Source Code Disclosure – CVE-2012-1823 (High Risk)

Vulnerability

Source Code Disclosure occurs when server-side application code becomes accessible to attackers.

Risk Level

- Risk: High
- Confidence: Medium

Description

Due to improper PHP CGI handling, the server may expose raw PHP source code instead of executing it.

Attackers can retrieve:

- Application logic
- Credentials
- Database connection strings
- API keys
- Hidden endpoints

Potential Sensitive Information

- MySQL usernames/passwords
- Admin credentials
- Session handling logic
- Hidden parameters
- File paths

CWE

CWE-200

Remediation

- Upgrade vulnerable PHP version
- Disable PHP CGI if unused
- Restrict direct access to sensitive files
- Remove backup/source files:
 - .bak
 - .old
 - .zip
- Configure proper Apache handlers
- Use least privilege permissions

6.3 Absence of Anti-CSRF Tokens (Medium Risk)

Vulnerability

Cross-Site Request Forgery (CSRF) allows attackers to force authenticated users to perform unwanted actions.

Risk Level

- Medium

Description

The application does not include CSRF protection tokens in sensitive requests.

Attack Scenario

An attacker tricks a logged-in victim into visiting a malicious page that silently sends requests to the vulnerable application.

Impact

- Password changes
- Unauthorized transactions
- Account manipulation
- Privilege changes

CWE

CWE-352

Remediation

- Add CSRF tokens to forms
- Validate tokens server-side
- Use Same Site cookie attribute
- Require re-authentication for critical actions
- Implement Origin/ Referer validation

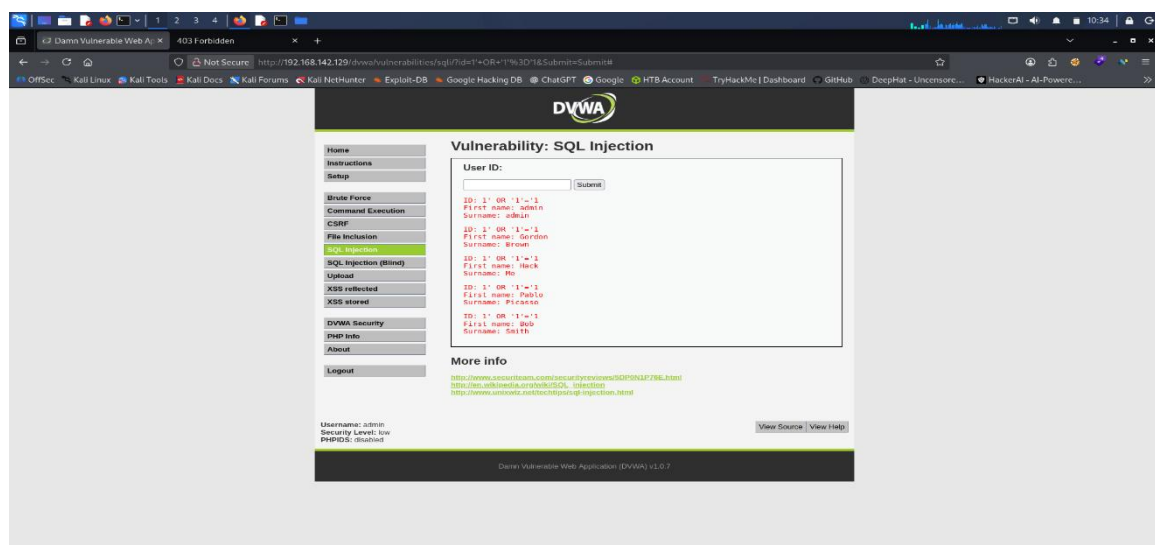
6.4 SQL Injection (Critical)

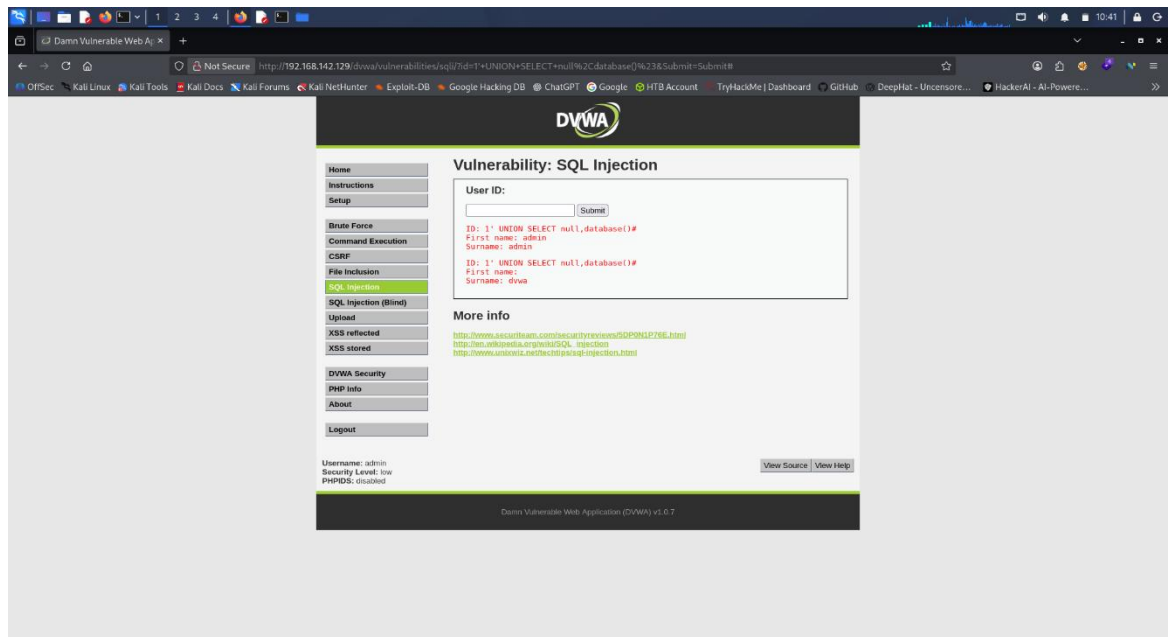
Vulnerability

- SQL Injection

Risk Level

- Critical
- Evidence from Screenshot





Payload used:

➤ `1' OR '1'='1`

Result:

Multiple database records displayed

Authentication/query bypass successful

Another payload:

➤ `1' UNION SELECT null,database()#`

Description

The application directly concatenates user input into SQL queries without proper sanitization.

Impact

An attacker can:

- Bypass authentication
- Dump database contents
- Extract usernames/passwords
- Modify or delete records
- Gain administrative access

CWE

- CWE-89

OWASP Category

- A03:2021 – Injection

Remediation

- Use prepared statements
- Implement parameterized queries
- Validate user input
- Apply least privilege DB accounts
- Use ORM frameworks securely

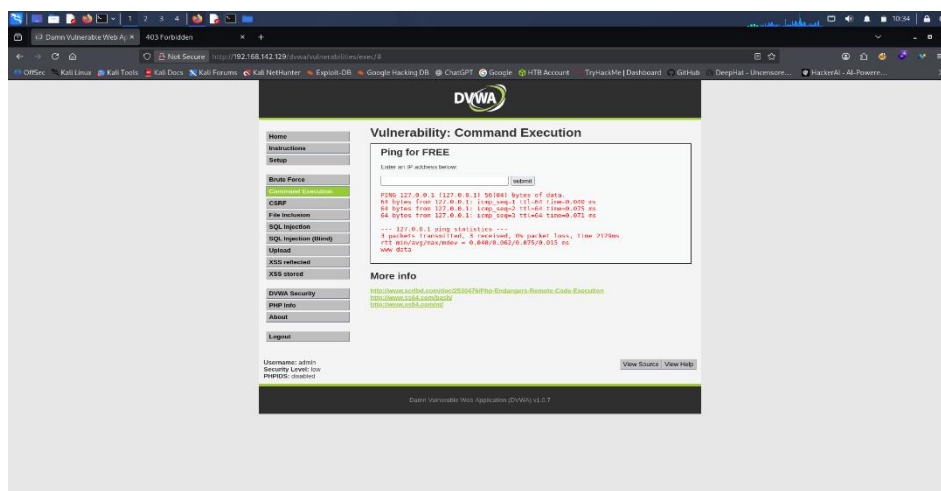
6.5 Command Injection / OS Command Execution (Critical)

Vulnerability

OS Command Injection

Risk Level

- Critical
- Evidence



Payload observed:

- 127.0.0.1 && whoami

Output:

- www-data

Description

The application executes user-controlled input in system shell commands.

Impact

An attacker can:

- Execute arbitrary commands
- Read/write server files
- Install malware
- Obtain reverse shells
- Fully compromise the server

CWE

- CWE-78

Remediation

- Avoid shell execution functions
- Use allowlists for input
- Escape shell arguments
- Run services with minimal privileges
- Disable dangerous PHP functions

6.6 Unrestricted File Upload (Critical)

Vulnerability

Unrestricted File Upload

Risk Level

- Critical
- Evidence

Uploaded file:

➤ code.desktop

Upload success message:

➤ successfully uploaded

Description

The application accepts uploaded files without proper validation of

- File type
- MIME type
- Extension
- Content

Impact

An attacker may:

- Upload web shells
- Execute malicious scripts
- Store malware
- Achieve remote code execution

CWE

- CWE-434

Remediation

- Allow only safe extensions
- Validate MIME types
- Rename uploaded files
- Store uploads outside web root
- Scan files for malware
- Disable script execution in upload directories

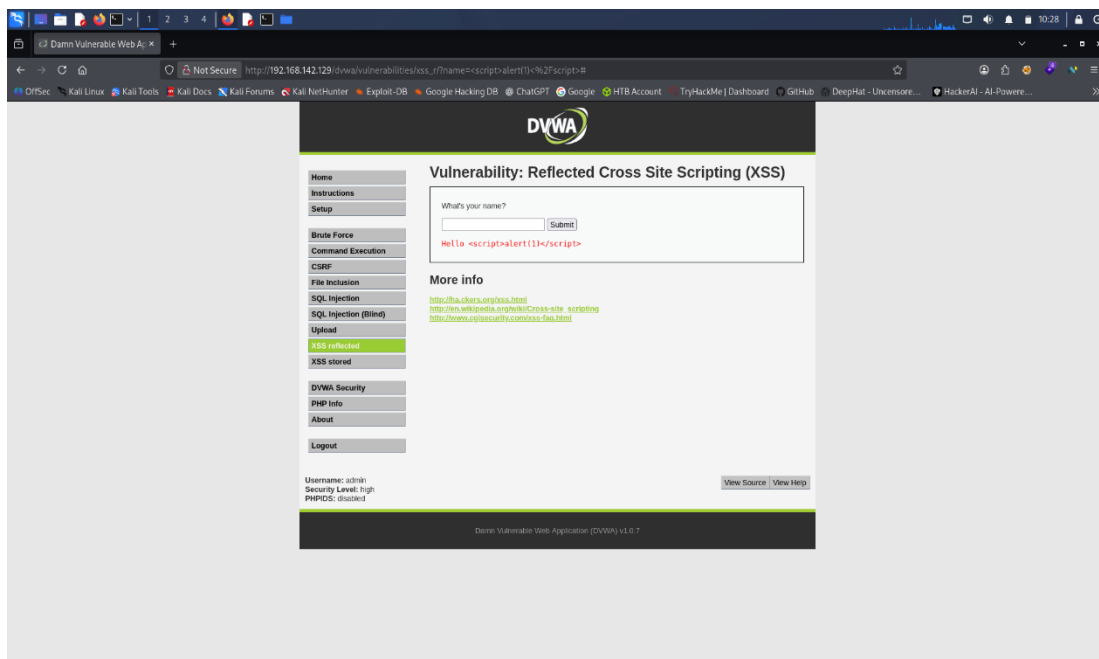
6.7 Reflected Cross-Site Scripting (XSS) (High)

Vulnerability

Cross-Site Scripting

Risk Level

- High
- Evidence



Payload:

- `<script>alert(1)</script>`

Application response:

- Hello `<script>alert(1)</script>`

Description

User input is reflected into the page without proper encoding or sanitization.

Impact

An attacker can:

- Steal session cookies
- Hijack accounts
- Redirect users
- Inject malicious JavaScript
- Perform phishing attacks

CWE

- CWE-79

Remediation

- Encode output properly
- Sanitize user input
- Implement CSP headers
- Use secure frameworks
- Validate allowed characters

Vulnerability	CVSS
SQL Injection	9.8
Command Injection	9.8
XSS	6.1
Missing Headers	4.3